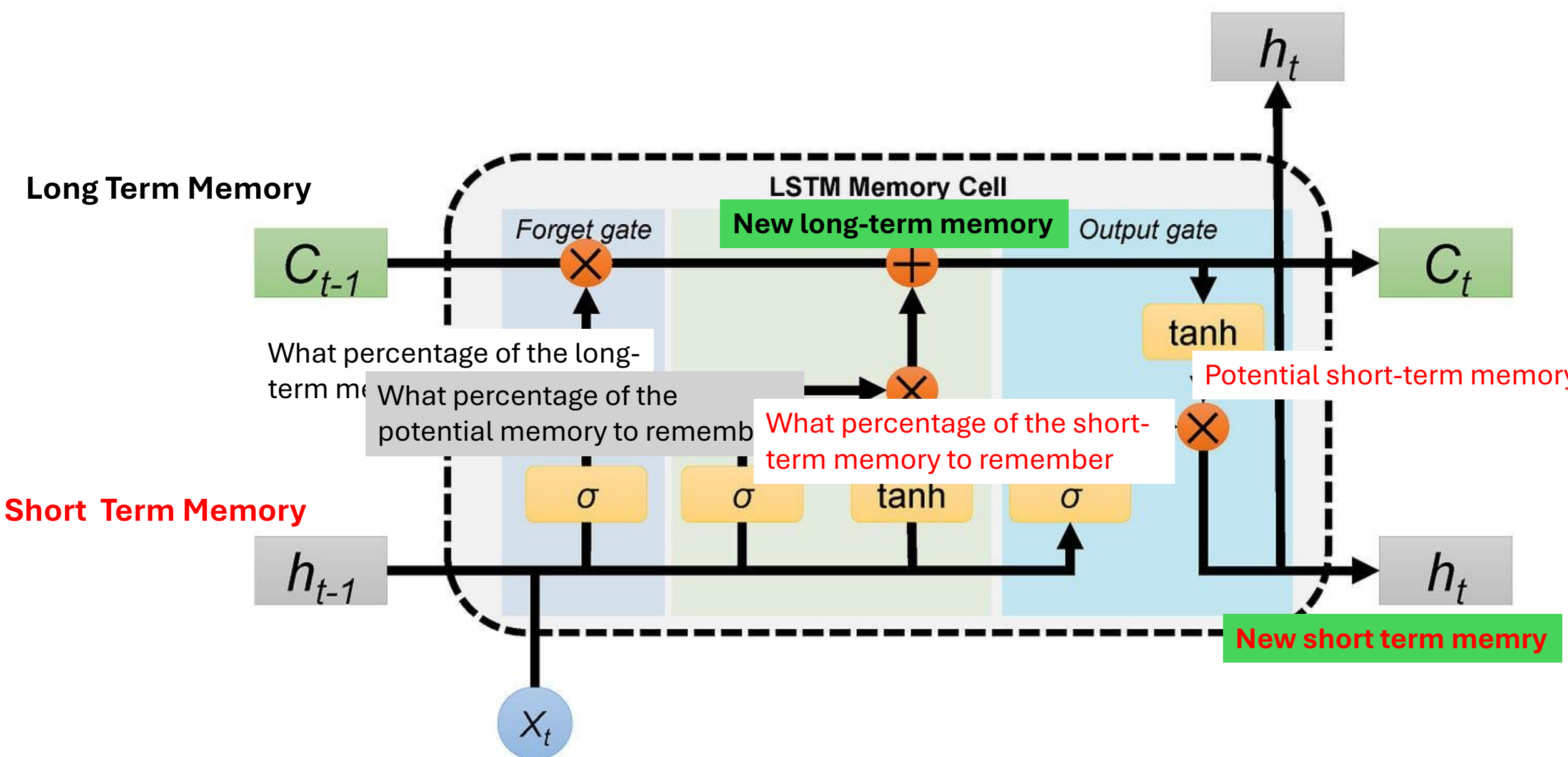




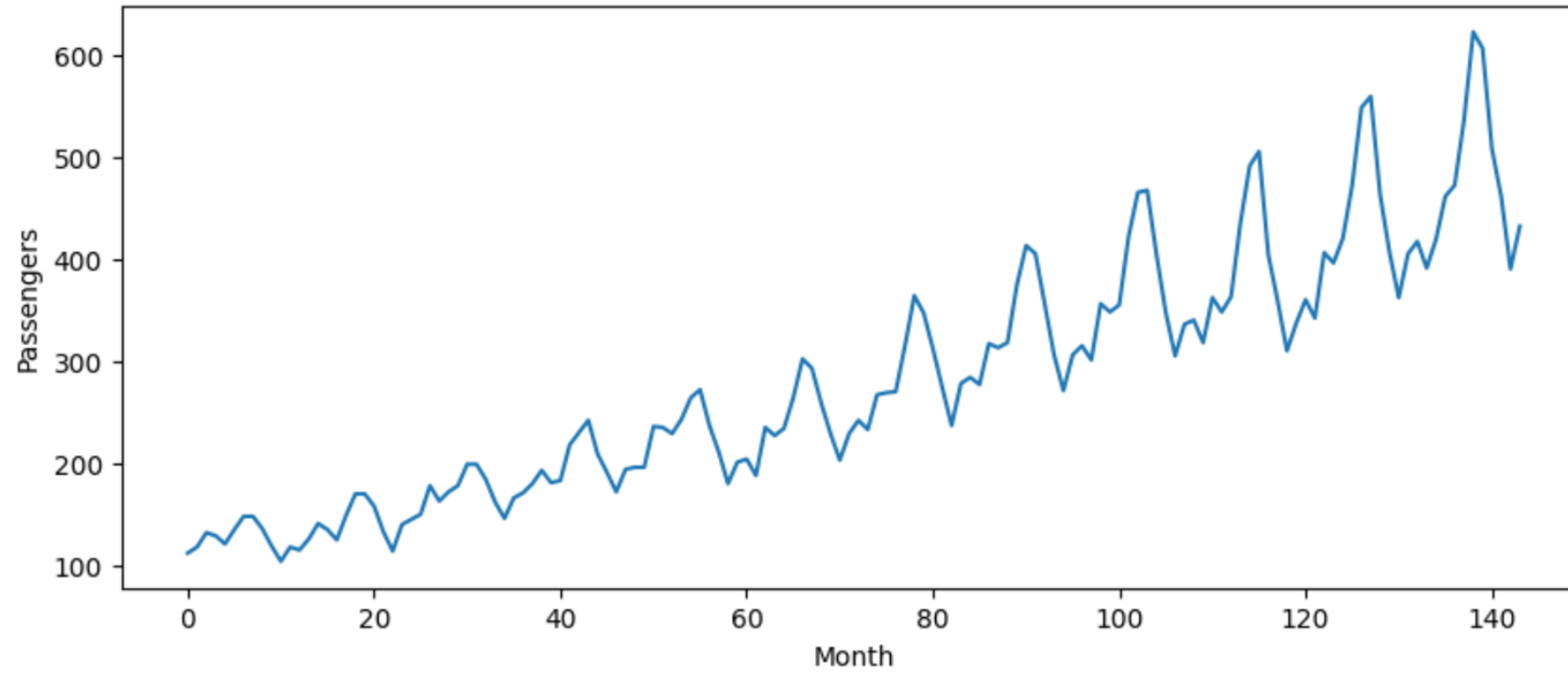
Predict the number of passenger next month

- Data set airline passengers :
- Load and visualize the dataset.
- Preprocess + normalize data.
- Prepare sequences for supervised learning.
- Build a **stacked LSTM model**.
- Train and evaluate.
- Plot **training history** and **forecast results**.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Load dataset
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv'
df = pd.read_csv(url, usecols=[1])
data = df.values.astype(float)
plt.figure(figsize=(10,4))
plt.plot(data)
plt.title('Monthly Airline Passengers')
plt.xlabel('Month')
plt.ylabel('Passengers')
plt.show()
```

Monthly Airline Passengers



```
# Normalize data
scaler = MinMaxScaler(feature_range=(0, 1))
data = scaler.fit_transform(data)

# Create supervised dataset
def create_dataset(dataset, look_back=1):
    X, Y = [], []
    for i in range(len(dataset)-look_back-1):
        X.append(dataset[i:(i+look_back), 0])
        Y.append(dataset[i + look_back, 0])
    return np.array(X), np.array(Y)

look_back = 12
X, y = create_dataset(data, look_back)
X = np.reshape(X, (X.shape[0], X.shape[1], 1))

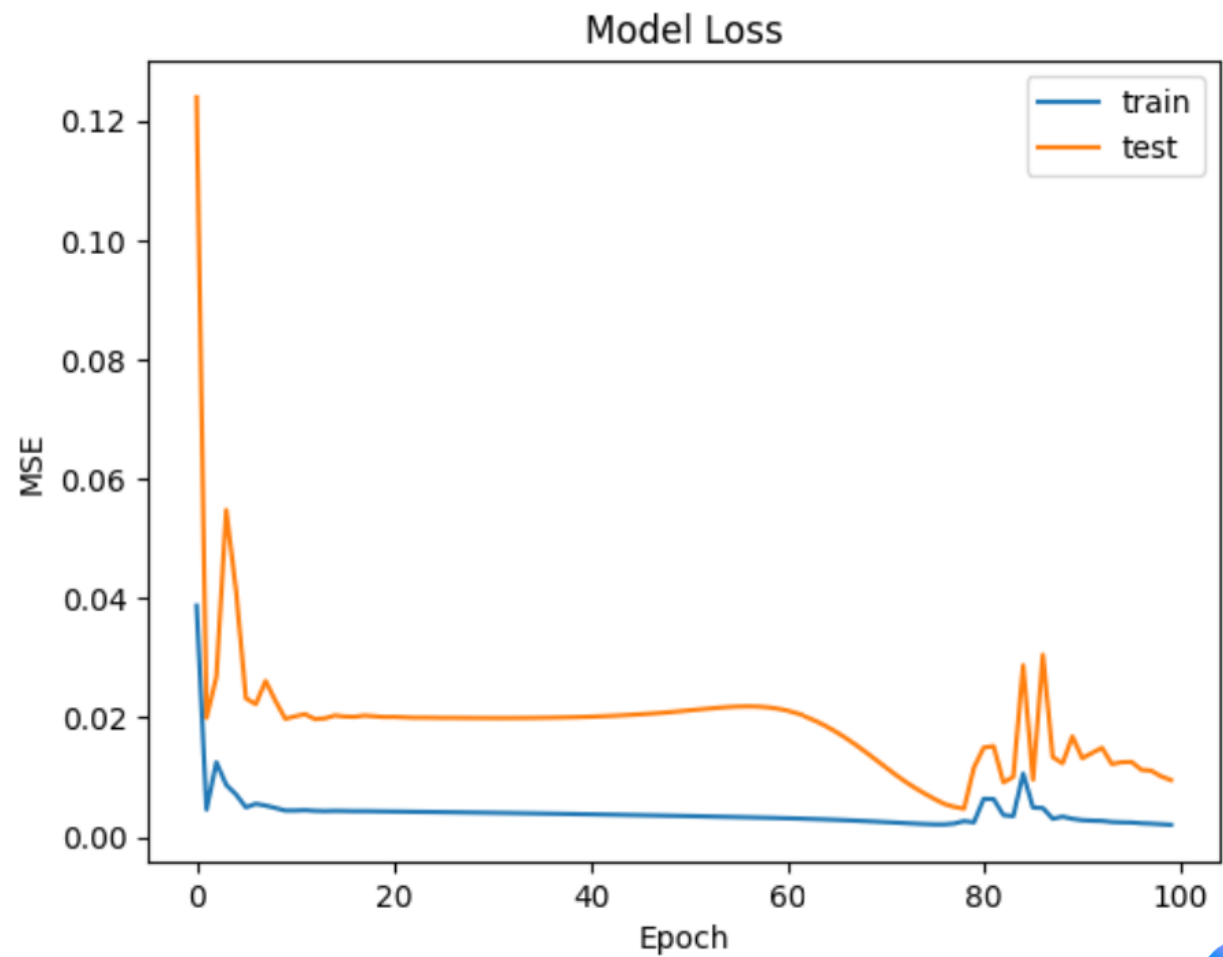
# Train/test split
train_size = int(len(X) * 0.67)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]
```

```
# Build model
model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(look_back, 1)),
    LSTM(50),
    Dense(1)
])
```

```
model.compile(optimizer='adam', loss='mse')
history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=16,
    validation_data=(X_test, y_test),
    verbose=1,
    shuffle=False
)
```

```
Epoch 4/100
6/6 ————— 0s 30ms/step - loss: 0.0081 - val_loss: 0.0548
Epoch 5/100
6/6 ————— 0s 26ms/step - loss: 0.0049 - val_loss: 0.0412
Epoch 6/100
```

```
# Plot training history
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='test')
plt.title('Model Loss')
plt.ylabel('MSE')
plt.xlabel('Epoch')
plt.legend()
plt.show()
```



```
# Predictions
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)

train_predict = scaler.inverse_transform(train_predict)
y_train_inv = scaler.inverse_transform([y_train])
test_predict = scaler.inverse_transform(test_predict)
y_test_inv = scaler.inverse_transform([y_test])

# Plot predictions
plt.figure(figsize=(12,6))
plt.plot(scaler.inverse_transform(data), label='True data')

# Shift train predictions
train_pred_plot = np.empty_like(data)
train_pred_plot[:, :] = np.nan
train_pred_plot[look_back:len(train_predict)+look_back, :] = train_predict

# Shift test predictions (FIXED index)
test_pred_plot = np.empty_like(data)
test_pred_plot[:, :] = np.nan
test_pred_plot[len(train_predict)+look_back:len(data)-1, :] = test_predict

plt.plot(train_pred_plot, label='Train predictions')
plt.plot(test_pred_plot, label='Test predictions')
plt.legend()
plt.title('Airline Passengers Forecast with LSTM')
plt.show()
```


Airline Passengers Forecast with LSTM

